

Sistema de Carpetas — Depto. Licencias de Conducir

Ilustre Municipalidad de Valparaíso — informe técnico

1. Qué es esto

El Excel **DETALLE CARPETAS DEPTO. LICENCIAS DE CONDUCIR 2026** era la agenda del departamento: 23 hojas, 21.568 filas, tres oficinas. Este sistema lo reemplaza por una aplicación web que corre en el mismo computador.

El Excel **se lee, nunca se modifica**. Se importa una vez y desde ahí la base de datos manda.

Pieza	Elección
Lenguaje	C# sobre .NET 10
Pantallas	Razor Pages (HTML generado en el servidor)
Base de datos	SQLite, un solo archivo
Lectura del Excel	ClosedXML
Pruebas	xUnit — 267 pruebas

2. Cómo fluye la información

El diagrama del final muestra el recorrido completo. En palabras:

1. El operador importa (por consola o desde la pantalla *Importar*).
2. El Excel se copia a un temporal **sin las listas desplegables**: ClosedXML se niega a abrir el original por culpa de ellas.
3. Cada hoja se clasifica **por sus encabezados**, no por su nombre.
4. Cada fila se valida y se convierte en un caso.
5. Todo se graba por lotes en SQLite; de ahí viven las pantallas y los informes.

3. El obstáculo que casi bloquea todo

ClosedXML no puede abrir el libro real: sus listas desplegables superan los 255 caracteres que acepta.

```
System.ArgumentOutOfRangeException: The maximum allowed length of the value is 255 characters.  
at ClosedXML.Excel.XLWorkbook.LoadDataValidations(...)
```

Esas validaciones no sirven para importar. La solución copia el archivo y les quita esos nodos, **sin tocar el original**.

src/LicenciasCarpetas/Import/WorkbookSanitizer.cs

```

/// <summary>Copies the workbook to a temp file and strips every data-validation element from it.
/// The caller owns the returned file and must delete it.</summary>
public static string CreateCopyWithoutDataValidations(string workbookPath)
{
    var tempPath = Path.Combine(Path.GetTempPath(), $"carpetas-import-{Guid.NewGuid():N}.xlsx");

    // Copied through a shared read stream: the workbook usually lives on a synced Drive folder
    // and may be open in Excel at the same time.
    using (var source = new FileStream(workbookPath, FileMode.Open, FileAccess.Read, FileShare.ReadWrite))
    using (var destination = new FileStream(tempPath, FileMode.CreateNew, FileAccess.Write,
    FileShare.None))
    {
        source.CopyTo(destination);
    }

    try
    {
        StripDataValidations(tempPath);
        return tempPath;
    }
    catch
    {
        File.Delete(tempPath);
        throw;
    }
}

private static void StripDataValidations(string xlsxPath)
{
    using var archive = ZipFile.Open(xlsxPath, ZipArchiveMode.Update);

    foreach (var entry in archive.Entries.ToList())
    {

```

4. Reconocer las hojas por su contenido

Al principio las hojas se buscaban por nombre. El operador renombró ESCANEADAS Y SUBIDAS a HOJA ESTADISTICAS y la importación dejó de traer 155 días de contadores **sin dar ningún error**. Ahora se clasifican por sus encabezados.

src/LicenciasCarpetas/Import/ExcelWorkbookImporter.cs

```

/// <summary>
/// Cada hoja se clasifica por sus encabezados, no por su nombre: el libro es de quien lo usa y
/// las hojas se renombran ("ESCANEADAS Y SUBIDAS" pasó a "HOJA ESTADISTICAS"). Atado al nombre,
/// el importador dejó de traer 149 días de contadores sin decir nada.
/// </summary>
public ImportSummary Import(IXLWorkbook workbook)
{
    var summary = new ImportSummary();

    foreach (var sheet in workbook.Worksheets)
    {
        // El nombre sigue mandando para la agenda: distingue oficina y mes, que los encabezados
        // no dicen, y descarta las hojas PLANTILLA con la misma estructura pero sin datos.
        if (AgendaSheet.TryParse(sheet.Name) is { } agenda && HasHeader(sheet, "FECHA DE LA CITACION"))
        {
            ImportAgenda(sheet, agenda, summary);
            continue;
        }

        if (HasHeader(sheet, "ESCANEADAS") && HasHeader(sheet, "SUBIDAS"))
        {
            ImportCounters(sheet, summary);
            continue;
        }

        if (HasHeader(sheet, "MUNICIPIO") && HasHeader(sheet, "CORREO"))
        {
            ImportDirectory(sheet, summary);
        }
    }

    return summary;
}

/// <summary>Busca un encabezado exacto en las primeras filas de la hoja.</summary>

```

5. Leer una fila: lo que dice y lo que significa

Una celda puede significar dos cosas. FECHA ULTIMA CARPETA trae una fecha (la carpeta está en Valparaíso) o el nombre de una comuna (hay que pedirla a otro municipio). Nada se descarta: lo que no se entiende se marca para revisión.

src/LicenciasCarpetas/Import/AgendaRowMapper.cs

```

/// <summary>Returns null when the row carries no person at all (spacer rows, trailing blanks).</summary>
public static FolderCase? Map(RawAgendaRow raw, AgendaSheet sheet, int rowNumber)
{
    var fullName = CellValue.ToText(raw.FullName);
    var firstName = CellValue.ToText(raw.FirstName);
    var lastName = CellValue.ToText(raw.LastName);
    var rawRut = CellValue.ToText(raw.Rut);

    if (fullName is null && firstName is null && lastName is null && rawRut is null)
    {
        return null;
    }

    // "NOMBRE COMPLETO" is a formula in the workbook and sometimes ends up empty even though the
    // two source columns are filled – rebuild it rather than leaving the row nameless.
    fullName ??= string.Join(' ', new[] { firstName, lastName }.Where(part => part is not null)).Trim();
    if (fullName.Length == 0)
    {
        fullName = null;
    }

    var validatedRut = RutValidator.NormalizeAndValidate(rawRut);
    var rawState = CellValue.ToText(raw.FolderState);
    var state = FolderStateCatalog.TryResolve(rawState);
    var rawDecision = CellValue.ToText(raw.FinalDecision);
    var decision = FinalDecisionCatalog.TryResolve(rawDecision);
    var attention = CellValue.ToText(raw.Attention);

    // Same cell, two meanings: a date means the folder is here in Valparaíso, a comuna name means
    // it has to be requested from that municipality (cambio de domicilio).
    var lastFolderDate = CellValue.ToDate(raw.LastFolder);
    var lastFolderComuna = lastFolderDate is null ? CellValue.ToText(raw.LastFolder) : null;

    // "SE ENCUENTRA EN ARCHIVOS" / "EN OF. 43" only repeat what the sector already says, and the
    // sector is derived from the date above. With a date present they are dropped, so importing
    // the workbook again cannot resurrect the labels the operator retired. Without a date they
    // are the only record of where the folder is, and they stay.
    if (lastFolderDate is not null && state is Domain.FolderState.SeEncuentraEnArchivos
        or Domain.FolderState.SeEncuentraEnOficina43)
    {
        state = null;
        rawState = null;
    }

    var folderCase = new FolderCase
    {
        CitationDate = CellValue.ToDate(raw.CitationDate),

```

6. El RUT se valida, no se cree

El dígito verificador se calcula. Un RUT inválido se conserva tal cual y el caso queda en

Requiere revisión: borrarlo sería perder lo que el operador escribió.

src/LicenciasCarpetas/Domain/RutValidator.cs

```

/// <summary>
/// Normalizes a Chilean RUT (with or without dots, upper/lowercase K) to canonical dotted form
/// (e.g. "18.785.387-7") and validates its check digit. Returns null if the input is not shaped
/// like a RUT or fails the check digit.
/// </summary>
public static string? NormalizeAndValidate(string? rawRut)
{
    if (rawRut is null)
    {
        return null;
    }

    var digitsAndK = new StringBuilder();
    foreach (var c in rawRut)
    {
        if (char.IsDigit(c) || c is 'k' or 'K')
        {
            digitsAndK.Append(char.ToUpperInvariant(c));
        }
    }

    if (digitsAndK.Length < 2)
    {
        return null;
    }

    var body = digitsAndK.ToString(0, digitsAndK.Length - 1);
    var checkDigit = digitsAndK[^1];

    if (body.Length is < 7 or > 8 || !body.All(char.IsDigit))
    {
        return null;
    }
}

```

7. El sector se deduce solo

Dónde está físicamente la carpeta no se elige a mano: sale de la fecha de la última carpeta.

Antes de julio 2023, Archivo; desde julio 2023, Oficina 43.

src/LicenciasCarpetas/Domain/FolderCase.cs

```

/// <summary>Where the physical folder is filed, derived from the última-carpeta date. Null while
/// there is no date (including cambio-de-domicilio rows, whose folder is in another comuna).</summary>
public FolderSector? Sector => LastFolderDate is { } fecha
    ? fecha < new DateOnly(2023, 7, 1) ? FolderSector.Archivo : FolderSector.Oficina43
    : null;

/// <summary>True when the folder was already uploaded to Conaset, whichever the upload route was.
</summary>

```

8. Los colores son los del Excel

El operador lee la agenda por color antes que por texto. Los 13 colores se copiaron de las reglas de formato condicional del propio libro. Única excepción: SUBIDA A CONASET, que el operador pidió en azul para separar el trabajo terminado.

src/LicenciasCarpetas/Domain/FolderState.cs

```

/// <summary>
/// Row colours taken from the workbook's own conditional formatting: the operator reads the
/// agenda by colour before reading any text, so these are copied exactly rather than redesigned.
/// "CANJE LIC. EXTRANJERA" has no rule in the workbook and therefore has no colour here either.
/// </summary>
private static readonly Dictionary<FolderState, string> Colors = new()
{
    [FolderState.PrimeraLicencia] = "#FF00FF",
    // El libro lo pintaba amarillo; el operador lo cambió a azul, porque una carpeta ya subida
    // a Conaset es trabajo terminado y debe distinguirse de un vistazo del resto.
    [FolderState.SubidaAConaset] = "#1155CC",
    [FolderState.SubidaConF8] = "#BF9000",
    [FolderState.SubidaConOficio] = "#FE599",
    [FolderState.CambioDomicilioSubidoAConaset] = "#00FFFF",
    [FolderState.CambioDomicilioSubidoConCorreo] = "#D0E0E3",
    [FolderState.CambioDomicilioSolicitado] = "#9FC5E8",
    [FolderState.CambioDomicilio] = "#3D85C6",
    [FolderState.SeEncuentraEnArchivos] = "#6AA84F",
    [FolderState.SeEncuentraEnOficina43] = "#8E7CC3",
    [FolderState.NoExisteCarpeta] = "#FF0000",
    [FolderState.CrearOficio] = "#C27BA0",
    [FolderState.CrearCertificado] = "#C27BA0"
};

/// <summary>
/// States retired from the dropdowns at the operator's request. They stay in the catalog on
/// purpose: cases imported from the 2026 workbook already carry them, and they must keep

```

9. Reimportar no duplica a nadie

Un caso es el mismo cuando coinciden oficina, fecha de citación y RUT. Si el RUT o la fecha no son legibles, la identidad es la celda de origen: hoja y fila.

src/LicenciasCarpetas/Persistence/FolderCaseRepository.cs

```

/// <summary>
/// Re-importing the same workbook must not duplicate anyone. A row is the same case when the
/// person, citation date and office match; when the RUT or the date is unreadable, the workbook
/// cell it came from (sheet + row) is used as identity instead.
/// </summary>
public UpsertOutcome Upsert(FolderCase folderCase)
{
    var existingId = FindExistingId(folderCase);
    if (existingId is null)
    {
        Insert(folderCase);
        return UpsertOutcome.Inserted;
    }

    using (var connection = Open())
    {
        Update(existingId.Value, folderCase, connection);
    }
    return UpsertOutcome.Updated;
}

```

10. Importar en lotes: de 87 a 15 segundos

Cada fila abría su propia conexión a SQLite. Agrupadas en una transacción por hoja, la importación completa del libro bajó de 87 a 15 segundos.

```

/// <summary>
/// Una conexión y una transacción para todo el lote. SQLite confirma cada escritura suelta en
/// disco; agrupadas, la importación completa del libro baja de minutos a segundos. Si algo
/// falla a mitad de camino, no queda media hoja importada.
/// </summary>
public IReadOnlyList<UpsertOutcome> UpsertMany(IReadOnlyList<FolderCase> folderCases)
{
    var outcomes = new List<UpsertOutcome>(folderCases.Count);
    if (folderCases.Count == 0)
    {
        return outcomes;
    }

    using var connection = Open();
    using var transaction = connection.BeginTransaction();

    foreach (var folderCase in folderCases)
    {
        var existingId = FindExistingId(folderCase, connection);
        if (existingId is null)
        {
            Insert(folderCase, connection);
            outcomes.Add(UpsertOutcome.Inserted);
        }
        else
        {
            Update(existingId.Value, folderCase, connection);
            outcomes.Add(UpsertOutcome.Updated);
        }
    }

    transaction.Commit();
    return outcomes;
}

```

11. Ordenar nombres chilenos

SQLite compara byte a byte: ÁLVARO y MUÑOZ caían después de la Z. Se guarda una copia del nombre sin tildes que solo sirve para ordenar; en pantalla se ve el nombre con sus tildes.

```

/// <summary>
/// Builds the ORDER BY from the closed <see cref="CaseSort"/> set – no caller text ever reaches
/// the SQL. Id breaks ties so paging can't show a row twice or skip one, and every column falls
/// back to the name so equal dates read alphabetically.
/// </summary>
private static string OrderBy(CaseFilter filter)
{
    // Dates read newest-first by default (that is how the agenda is consulted); text reads A-Z.
    var (column, descendingByDefault) = filter.Sort switch
    {
        // FullNameSort, not FullName: SQLite compares bytes, so ÁLVARO and MUÑOZ would land
        // after Z on the raw text.
        CaseSort.Name => ("FullNameSort COLLATE NOCASE", false),
        CaseSort.Rut => ("Rut", false),
        CaseSort.Office => ("Office", false),
        CaseSort.FolderUploadedDate => ("FolderUploadedDate", true),
        CaseSort.LastFolderDate => ("LastFolderDate", false),
        CaseSort.FolderState => ("FolderState", false),
        CaseSort.FinalDecision => ("FinalDecision", false),
        _ => ("CitationDate", true)
    };

    var direction = filter.Descending != descendingByDefault ? "DESC" : "ASC";

    // En la vista por defecto, lo ya subido a Conaset es trabajo terminado: baja al final de la
    // lista y entre ellos van en orden de subida. Si el operador pide una columna concreta,
    // manda esa columna y no se altera nada.
    if (filter.Sort == CaseSort.CitationDate)
    {
        var conaset = (int)Domain.FolderState.SubidaAConaset;
        return $""
            CASE WHEN FolderState = {conaset} THEN 1 ELSE 0 END ASC,
            CASE WHEN FolderState = {conaset} THEN FolderUploadedDate END ASC,
            {column} {direction}, FullNameSort COLLATE NOCASE ASC, Id ASC
            "";
    }
}

```

12. Nada se borra de verdad

Eliminar manda el caso a la papelera: desaparece de listados, estadísticas y exportaciones, una reimportación no lo revive, y se puede restaurar. El borrado definitivo solo existe dentro de la papelera.

src/LicenciasCarpetas/Persistence/FolderCaseRepository.cs

```

public void Delete(long id)
{
    using var connection = Open();
    using var command = connection.CreateCommand();
    command.CommandText = "UPDATE FolderCase SET DeletedAt = $deletedAt WHERE Id = $id";
    command.Parameters.AddWithValue("$deletedAt", DateTimeOffset.UtcNow.ToString("O"));
    command.Parameters.AddWithValue("$id", id);
    command.ExecuteNonQuery();
}

public void Restore(long id)

```

13. Respaldo en cada arranque

Todo vive en un archivo de 8 MB. Cada arranque lo copia **antes** de aplicar migraciones y conserva las últimas diez. Si el respaldo falla, la aplicación arranca igual: un problema de copia

nunca puede dejar al operador sin trabajar.

src/LicenciasCarpetas/Persistence/DatabaseBackup.cs

```
/// <summary>
/// Returns the path of the copy, or null when there was nothing to copy or the copy failed –
/// a backup problem is never a reason to stop the operator from working.
/// </summary>
public string? Run(DateTimeOffset now)
{
    try
    {
        if (!File.Exists(databasePath))
        {
            return null;
        }

        Directory.CreateDirectory(backupDirectory);

        var name = $"{Path.GetFileNameWithoutExtension(databasePath)}-{now:yyyyMMdd-HH:mm}.db";
        var destination = Path.Combine(backupDirectory, name);

        // Restarting the app twice within the same minute must not pile up identical copies.
        if (!File.Exists(destination))
        {
            File.Copy(databasePath, destination);
        }

        RemoveOldCopies();
        return destination;
    }
    catch (Exception)
    {
        return null;
    }
}

private void RemoveOldCopies()
```

14. Contraseñas

PBKDF2 con sal por usuario y 210.000 iteraciones. Cinco intentos fallidos bloquean la cuenta quince minutos. No hay recuperación por correo: sin servidor de correo, una pantalla que cambiara contraseñas sin sesión iniciada entregaría la agenda a cualquiera que alcance el puerto.

src/LicenciasCarpetas/Dashboard/Auth/PasswordHasher.cs

```
public static (string Hash, string Salt, int Iterations) Hash(string password)
{
    var salt = RandomNumberGenerator.GetBytes(SaltSizeBytes);
    var hash = Rfc2898DeriveBytes.Pbkdf2(password, salt, DefaultIterations, HashAlgorithmName.SHA256, HashSizeBytes);
    return (Convert.ToBase64String(hash), Convert.ToBase64String(salt), DefaultIterations);
}

public static bool Verify(string password, string storedHash, string storedSalt, int iterations)
{
    var salt = Convert.FromBase64String(storedSalt);
    var expected = Convert.FromBase64String(storedHash);
    var actual = Rfc2898DeriveBytes.Pbkdf2(password, salt, iterations, HashAlgorithmName.SHA256, expected.Length);
```

15. Estadísticas: lo calculado y lo escrito

Agendadas, atendidas y porcentaje de atención **se cuentan solos** desde los casos. Solo escaneadas y subidas siguen siendo contadores manuales: no se pueden deducir de la agenda.

src/LicenciasCarpetas/Statistics/StatisticsService.cs

```
public MonthlyStatistics ForMonth(int year, int month)
{
    var attendance = cases.DailyAttendance(year, month);
    var monthCounters = counters.ForMonth(year, month).ToDictionary(counter => counter.Date);

    var dates = attendance
        .Select(entry => entry.Date)
        .Concat(monthCounters.Keys)
        .Distinct()
        .OrderBy(date => date)
        .ToList();

    var days = new List<DailyStatisticsRow>(dates.Count);
    foreach (var date in dates)
    {
        var byOffice = attendance
            .Where(entry => entry.Date == date)
            .ToDictionary(entry => entry.Office, entry => (entry.Scheduled, entry.Attended));

        monthCounters.TryGetValue(date, out var counter);
        days.Add(new DailyStatisticsRow(date, counter?.Scanned, counter?.Uploaded, byOffice));
    }

    return new MonthlyStatistics(
        year,
        month,
        days,
        cases.FolderStateBreakdown(year, month, office: null),
        cases.FinalDecisionBreakdown(year, month, office: null),
        cases.LicenceClassBreakdown(year, month, office: null));
}
```

16. Clases de licencia

Una citación puede cubrir varias clases (B y C, o A2 y A3). Se guardan como texto ordenado (B,C) y en las estadísticas cada clase suma por separado: la pregunta es cuántas licencias se tramitan, no cuántas personas vinieron.

src/LicenciasCarpetas/Domain/LicenceClass.cs

```

/// <summary>
/// Lee la selección guardada ("B,C"). Un código desconocido se ignora en vez de reventar: la
/// columna es texto y puede traer restos de una versión anterior o de una edición a mano.
/// </summary>
public static IReadOnlyList<LicenceClass> Parse(string? stored)
{
    if (string.IsNullOrEmpty(stored))
    {
        return [];
    }

    var selected = new List<LicenceClass>();
    foreach (var piece in stored.Split(',', StringSplitOptions.RemoveEmptyEntries |
StringSplitOptions.TrimEntries))
    {
        if (Enum.TryParse<LicenceClass>(piece, ignoreCase: true, out var licence) &&
!selected.Contains(licence))
        {
            selected.Add(licence);
        }
    }

    selected.Sort();
    return selected;
}

/// <summary>Guarda en orden de catálogo y sin repetidos, para que dos selecciones iguales se
/// escriban igual y las estadísticas puedan agrupar sin sorpresas.</summary>
public static string? Serialize(IEnumerable<LicenceClass> selection)
{

```

17. Diagrama del sistema

